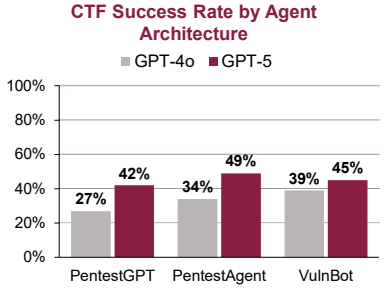
FAILURE-AWARE EPISODIC MEMORY FOR PENETRATION-TESTING AGENTS

Shrikant Bhatnagar; Dr. Keke Chen, Ph.D. (Email: vt06290@umbc.edu)
Department of Computer Science and Engineering

Research problem

- Penetration testing, a practice of probing systems for exploitable weaknesses before real attackers do, is essential to securing software, but it is labour-intensive and requires scarce expertise: ISC2 estimates a global shortfall of roughly 4.7 million cybersecurity professionals.
- Large language models can now attempt penetration testing autonomously, promising to reduce assessment cost and make advanced testing accessible to organizations that can't afford dedicated expertise.



- Current agents, however, remain immature. On benchmarks they solve only easy and moderate difficulty challenges.
- Existing architectures treat failure as something to reflect on in the moment rather than knowledge to be retained and reused.

Can failure-aware structured episodic memory reduce repeated ineffective exploration and improve the performance of autonomous penetration-testing agents?

Background

LLM Agents:

An LLM becomes an agent when it is given tools (shell access, APIs etc) and a scaffold that loops: propose a command, run it, read the output, decide the next move. Unlike a chatbot, which answers once and waits, an agent executes against the live target and adapts over dozens of steps without a human in the loop.

Penetration-Testing:

Penetration testing is the practice of simulating an attack against a system, with authorization, in order to find vulnerabilities before a real attacker does.

A typical penetration testing task follows a loose cycle:

1. Reconnaissance Map the target to find open ports and exposed files → **2. Enumeration** Probe those services for specific weaknesses → **3. Exploitation** Use a weakness to gain access or extract information

repeat as new services and weaknesses surface

Capture-the-Flag:

A Capture-the-Flag (CTF) challenges is a deliberately vulnerable system containing a hidden string called a flag. Recovering the flag is proof that the system was compromised, which makes CTFs one of the few security tasks with an automatically checkable success signal. This project uses challenges drawn from the **Cybench** benchmark suite.

Methodology

1. Baseline:

The **PentestGPT loop** was used as the baseline and the injection surface. While its reasoning module is able to keep a track of the penetration testing activity, it does not track failures.

The PentestGPT architecture uses the following loop:

- **Reasoning:** reads task state from the history and picks the next sub-task
- **Generation:** writes the concrete command for that sub-task
- **Parsing:** reads the output, updates state; the loop repeats every turn

2. Memory:

Writing:

- A failed turn's command and tool output is scored by a deterministic evaluator. A failure triggers one structured LLM call returning {technique_class, reflection}, stored as an *attempt* episode.

Turn output command + tool output → **Evaluate failure** Check for failure → **classify + reflect** 1 LLM call, class + reflection

Injecting:

Failure Memory Block
1. **Dead-End Warnings**: technique classes filtered to the current phase;
2. **Last 5 Failed Attempts** with one-line reflections

→ **Reasoning: strategic use** stop re-selecting an exhausted technique

→ **Generation: tactical use** fix the specific command

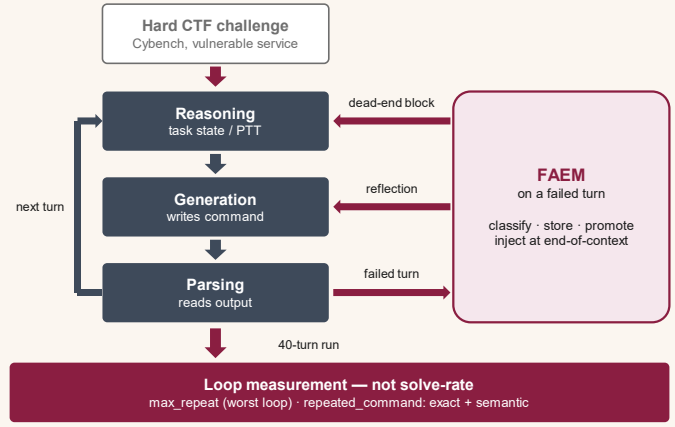
3. Challenge Runs:

Hard tasks were chosen: Loops only show up when the task is hard enough to defeat the agent's first attempts.

These tasks were run on two different arms with 40 turns each:

1. **Baseline Arm**: The PentestGPT agent with no memory. It measures how much the agent loops when nothing intervenes
2. **FAEM Arm**: The agent with failure-aware memory. FAEM is the *only* thing that differs between the arms, so any drop in looping is attributable to it.

Testing Framework



Mechanisms

Failure-Aware Episodic Memory:

- FAEM Stores failed attempts, dead-ends, and unsuccessful reasoning as structured, retrievable memory, so the agent stops re-trying what already failed.
- Attributes failed attempts with reflections on why the commands failed.

Loop Detection:

- A loop is the same/near-identical command re-issued across turns without making progress.
- Every command the agent runs is logged and compared against the earlier commands in that run.
- The single most-repeated command's count is *max_repeat*.
- Near-identical retries are also caught by embedding similarity (cosine ≥ 0.85), not just exact matches.

Results

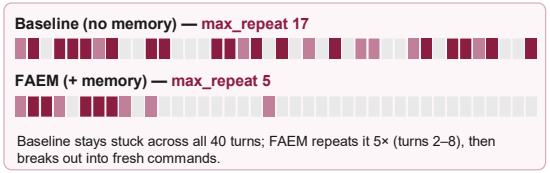
1. Reduction in Length of Worst loop in challenges:

Challenge (category)	n	Number of loops		Δ
		PentestGPT	FAEM	
Permuted (crypto)	3	12.7	6	−53%
network-tools (pwn)	1	6	2	−67%
Crushing (rev)	1	7	2	−71%

2. Loop Visualization

One Permuted run per arm, turns 1 → 40 left to right.

- **Dark** = most-repeated command (broken heredoc)
- mid = other repeats; light = one-off commands.



Conclusion and Future Work

- Autonomous pen-test agents waste much of their limited turn budget re-issuing commands that have already failed.
- FAEM (a memory of the agent's own failures) cuts the worst loop by **53–71%** across three hard challenges with memory the only difference between arms.
- The strongest result replicates across 3 seeds, and the gain is behavioural: less wasted exploration, measured directly rather than inferred.

Acknowledgments

This research was conducted in the Trustworthy and Intelligent Computing lab under the COEIT Summer Student Program (CSSP).